

## 📖 Programación en C (Repaso)

Sitio: [GRADO 25-26](#)  
Curso: SISTEMAS OPERATIVOS - 2526 (COMÚN)  
Libro: 📖 Programación en C (Repaso)

Imprimido por: GARCIA ZAFRA, ALVARO  
Día: domingo, 14 de diciembre de 2025, 10:29

## Tabla de contenidos

### 1. Programación en C

#### 1.1. Cadenas de texto en C

#### 1.2. Macros

#### 1.3. Máscaras

#### 1.4. Punteros a funciones (y callbacks)

### 2. Makefile

# I. Programación en C

Estos sub-capítulos tienen como objetivo repasar el lenguaje de programación C, especialmente aquello que difiere con C++ o que son especialmente complicadas si no se dominan los lenguajes a bajo nivel.

**¡Ojo! ¡Esta parte se asume que el estudiante la conoce previamente! Por lo que perfectamente puede caer en ejercicios, de hecho, el guion de prácticas asume que se conoce.**

## I.I. Cadenas de texto en C

# Cadenas de texto en C

En C, las cadenas de texto son una secuencia de caracteres que se almacenan en memoria. A diferencia de otros lenguajes de programación, C no tiene un tipo de dato específico para cadenas de texto. En su lugar, se utilizan *arrays* de caracteres y punteros para manipular las cadenas.

## Cadenas de texto como *arrays* de caracteres

Una cadena de texto en C es un *array* de caracteres que termina con un carácter nulo (`\0`). Por ejemplo:

```
char mi_cadena[] = "Hola";
```

En este caso, `mi_cadena` es un *array* de caracteres que contiene los siguientes valores:

- `mi_cadena[0]`: 'H'
- `mi_cadena[1]`: 'o'
- `mi_cadena[2]`: 'l'
- `mi_cadena[3]`: 'a'
- `mi_cadena[4]`: `\0` (carácter nulo)

El carácter nulo (`\0`) indica el final de la cadena. Sin él, el programa no sabría dónde termina la cadena y continuaría leyendo hasta encontrar el próximo `\0`, muy probablemente fuera de la memoria reservada (lo que suele llevar a errores de tipo `Segmentation Fault` o a comportamientos impredecibles en el programa).

## Cadenas de texto como punteros

Una cadena de texto también puede ser representada como un puntero a un *array* de caracteres. Por ejemplo:

```
char *mi_cadena = "Hola";
```

En este caso, `mi_cadena` es un puntero que apunta al primer elemento del *array* de caracteres. El *array* de caracteres es creado automáticamente por el compilador y se almacena en memoria.

## Diferencias entre cadenas de texto literales y variables

Hay una diferencia importante entre cadenas de texto literales (como `"Hola"`) y variables de tipo "cadena de texto" (como `mi_cadena`).

Las cadenas de texto literales se almacenan en memoria solo una vez, y todas las variables que las utilizan apuntan a la misma ubicación en memoria.

Por otro lado, las variables de cadena de texto se almacenan en memoria cada vez que se crean, y cada variable tiene su propia copia de la cadena.

## Manipulación de cadenas de texto

Para manipular cadenas de texto en C, se pueden utilizar las siguientes funciones:

- `strlen`: devuelve la longitud de una cadena de texto
- `strcpy`: copia una cadena de texto en otra
- `strcat`: concatena dos cadenas de texto
- `strcmp`: compara dos cadenas de texto

Por ejemplo:

```
#include <stdio.h>
#include <string.h>

int main() {
    char mi_cadena[] = "Hola";
    char otra_cadena[] = " mundo";

    printf("Longitud de mi_cadena: %d\n", strlen(mi_cadena));
    strcpy(mi_cadena, otra_cadena);
    printf("mi_cadena ahora es: %s\n", mi_cadena);
    strcat(mi_cadena, "!");
    printf("mi_cadena ahora es: %s\n", mi_cadena);

    return 0;
}
```

# Función `printf`

`printf` es una función en C que se utiliza para imprimir texto y valores en la consola. Su sintaxis básica es:

```
printf(formato, argumento1, argumento2,...);
```

Donde `formato` es una cadena de texto que contiene especificadores de formato, y `argumento1`, `argumento2`, etc. son los valores que se van a imprimir.

## Especificadores de formato

Los especificadores de formato son caracteres especiales que se utilizan en la cadena de formato para indicar el tipo de valor que se va a imprimir. A continuación, se presentan algunos de los especificadores de formato más comunes:

- `%i`: imprime un entero (int)
- `%d`: imprime un entero decimal (int)
- `%u`: imprime un entero sin signo (unsigned int)
- `%s`: imprime una cadena de texto (char \*)
- `%c`: imprime un carácter (char)
- `%f`: imprime un número flotante (float)
- `%e`: imprime un número flotante en notación científica (float)
- `%x`: imprime un entero en hexadecimal (int)
- `%o`: imprime un entero en octal (int)

### Ejemplos

A continuación, se presentan algunos ejemplos de uso de `printf` con diferentes especificadores de formato:

```
#include <stdio.h>

int main() {
    int entero = 10;
    float flotante = 3.14;
    char caracter = 'A';
    char cadena[] = "Hola";

    printf("Entero: %i\n", entero);
    printf("Flotante: %f\n", flotante);
    printf("Caracter: %c\n", caracter);
    printf("Cadena: %s\n", cadena);

    return 0;
}
```

## Especificadores de formato con precisión

Es posible especificar la precisión de los valores flotantes utilizando el especificador de formato `%.xf`, donde `x` es el número de decimales que se van a imprimir. Por ejemplo:

```
float flotante = 3.14159265359;

printf("Flotante con 2 decimales: %.2f\n", flotante);
printf("Flotante con 5 decimales: %.5f\n", flotante);
```

## Especificadores de formato con ancho

Es posible especificar el ancho del campo de impresión utilizando el especificador de formato `%xy`, donde `x` es el ancho del campo y `y` es el especificador de formato. Por ejemplo:

```
int entero = 10;

printf("Entero con ancho 5: %5i\n", entero);
printf("Entero con ancho 10: %10i\n", entero);
```

## I.2. Macros

# Macros

En C, las macros son una forma de definir una secuencia de código que se puede reutilizar en diferentes partes del programa. Una macro es una instrucción que se reemplaza por una secuencia de código en tiempo de compilación.

## Definición de una macro

La forma general de definir una macro es:

```
#define NOMBRE_MACRO argumentos
```

Donde **NOMBRE\_MACRO** es el nombre de la macro y **argumentos** es la secuencia de código que se reemplazará.

## Macros sin parámetros

Un ejemplo de macro sin parámetros es:

```
#define PI 3.14
```

Esta macro reemplaza la constante **PI** por el valor **3.14** en cualquier parte del código donde se utilice.

## Macros con parámetros

Una macro con parámetros se define de la siguiente manera:

```
#define NOMBRE_MACRO(par1, par2, ...) secuencia de código
```

Donde **par1**, **par2**, etc. son los parámetros de la macro y **secuencia de código** es la secuencia de código que se reemplazará.

Un ejemplo de macro con parámetros es:

```
#define SUMA(a, b) a + b
```

Esta macro reemplaza la expresión **SUMA(a, b)** por la suma de **a** y **b** en cualquier parte del código donde se utilice.

## Uso de macros con parámetros

Para utilizar una macro con parámetros, se debe llamar a la macro con los parámetros correspondientes. Por ejemplo:

```
int resultado = SUMA(2, 3);
```

Esta línea de código se reemplaza por:

```
int resultado = 2 + 3;
```

## Ejemplos de macros con parámetros

Aquí te dejo algunos ejemplos de macros con parámetros:

- Macro para intercambiar dos variables:

```
#define INTERCAMBIA(a, b) { int temp = a; a = b; b = temp; }
```

- Macro para calcular el área de un rectángulo:

```
#define AREA_RECTANGULO(ancho, alto) ancho * alto
```

- Macro para calcular el perímetro de un círculo:

```
#define PERIMETRO_CIRCULO(radio) 2 * 3.14 * radio
```

# Macros de inclusión

Cuando se trabaja con archivos de cabecera (**.h**) en C, es común que se incluyan múltiples veces en diferentes partes del código. Esto puede causar problemas de redefinición de funciones y variables, lo que puede llevar a errores de compilación.

La utilización de macros de inclusión ofrece varias ventajas:

- Evita la inclusión múltiple de archivos de cabecera

- Evita la redefinición de funciones y variables
- Mejora la eficiencia del compilador
- Facilita la depuración del código

### La solución: macros de inclusión

Una forma de evitar la inclusión múltiple de archivos de cabecera es utilizar macros de inclusión. Estas macros se utilizan para verificar si un archivo de cabecera ya ha sido incluido, y si es así, no lo incluye de nuevo.

### La forma general

La forma general de utilizar macros de inclusión es la siguiente:

```
#ifndef NOMBRE_MACRO
#define NOMBRE_MACRO

// Contenido del archivo de cabecera

#endif
```

Donde **NOMBRE\_MACRO** es el nombre de la macro de inclusión.

### Ejemplo

Supongamos que tenemos un archivo de cabecera llamado **mi\_cabecera.h** que contiene la siguiente definición:

```
#ifndef MI_CABECERA_H
#define MI_CABECERA_H

// Contenido del archivo de cabecera
void mi_funcion(void);

#endif
```

### Cómo funciona

Cuando se incluye el archivo **mi\_cabecera.h** por primera vez, la macro **MI\_CABECERA\_H** no está definida, por lo que se define y se incluye el contenido del archivo de cabecera.

Si se incluye el archivo **mi\_cabecera.h** de nuevo en otro lugar del código, la macro **MI\_CABECERA\_H** ya está definida, por lo que no se incluye el contenido del archivo de cabecera de nuevo.

## I.3. Máscaras

# Operadores binarios

En C, los operadores binarios o *bitwise* se utilizan para realizar operaciones a nivel de bits sobre variables enteras. Una máscara es un valor entero que se utiliza para seleccionar o modificar ciertos bits de una variable.

## Operadores *bitwise*

Los operadores *bitwise* en C son:

- **&** (AND)
- **|** (OR)
- **^** (XOR)
- **~** (NOT)
- **<<** (desplazamiento a la izquierda)
- **>>** (desplazamiento a la derecha)

## Máscaras

Una máscara es un valor entero que se utiliza para seleccionar o modificar ciertos bits de una variable. Por ejemplo, si tenemos una variable **x** de 8 bits y queremos seleccionar solo los 4 bits más significativos, podemos utilizar la máscara **0xF0**. Fíjate que en hexadecimal, ese valor corresponde en binario al número **11110000**.

### Ejemplos

1. **AND (&)**: selecciona solo los bits que están en ambos operandos.

```
int x = 0x12; // 00010010
int mascara = 0xF0; // 11110000
int resultado = x & mascara; // 00010000
```

2. **OR (|)**: selecciona todos los bits que están en alguno de los operandos.

```
int x = 0x12; // 00010010
int mascara = 0x0F; // 00001111
int resultado = x | mascara; // 00011111
```

3. **XOR (^)**: selecciona solo los bits que están en uno de los operandos, pero no en ambos.

```
int x = 0x12; // 00010010
int mascara = 0x0F; // 00001111
int resultado = x ^ mascara; // 00011101
```

4. **NOT (~)**: invierte todos los bits del operando.

```
int x = 0x12; // 00010010
int resultado = ~x; // 11101101
```

5. **Desplazamiento a la izquierda (<<)**: desplaza los bits del operando a la izquierda y rellena con ceros a la derecha.

```
int x = 0x12; // 00010010
int resultado = x << 2; // 01001000
```

6. **Desplazamiento a la derecha (>>)**: desplaza los bits del operando a la derecha y rellena con ceros a la izquierda.

```
int x = 0x12; // 00010010
int resultado = x >> 2; // 00000101
```

## I.4. Punteros a funciones (y callbacks)

# Punteros a funciones

Ya sabes que una función es un bloque de código que se puede llamar varias veces desde diferentes partes del programa. Pero, ¿qué pasa si queremos pasar una función como argumento a otra función? ¿O si queremos devolver una función desde otra función? Esto es posible gracias a los punteros a funciones. Esto se relaciona con los callbacks que verás más adelante en este tutorial.

## ¿Qué es un puntero a función?

Un puntero a función es una variable que almacena la dirección de memoria de una función. Al igual que un puntero a variable, un puntero a función es una variable que contiene la dirección de memoria de algo, pero en este caso, es la dirección de una función.

## Declaración de un puntero a función

La declaración de un puntero a función es similar a la declaración de un puntero a variable, pero con algunas diferencias. La sintaxis general es:

```
tipo_de_retorno (*nombre_puntero)(tipo_de_argumento1, tipo_de_argumento2, ...);
```

Donde:

- `tipo_de_retorno` es el tipo de valor que devuelve la función.
- `nombre_puntero` es el nombre del puntero a función.
- `tipo_de_argumento1`, `tipo_de_argumento2`, etc. son los tipos de los argumentos que recibe la función.

### Ejemplo

Supongamos que tenemos una función `suma` que recibe dos enteros y devuelve un entero:

```
int suma(int a, int b) {  
    return a + b;  
}
```

Podemos declarar un puntero a función que apunte a la función `suma` de la siguiente manera:

```
int (*puntero_suma)(int, int) = suma;
```

## Uso de un puntero a función

Una vez que tenemos un puntero a función, podemos utilizarlo para llamar a la función que apunta. La sintaxis es similar a la de llamar a una función normal, pero debemos utilizar el operador de llamada a función `()` después del nombre del puntero:

```
int resultado = (*puntero_suma)(2, 3);
```

## Pasar un puntero a función como argumento

Podemos pasar un puntero a función como argumento a otra función. Por ejemplo:

```
void imprimir_resultado(int (*puntero_funcion)(int, int), int a, int b) {  
    int resultado = (*puntero_funcion)(a, b);  
    printf("El resultado es: %d\n", resultado);  
}  
  
int main() {  
    int (*puntero_suma)(int, int) = suma;  
    imprimir_resultado(puntero_suma, 2, 3);  
    return 0;  
}
```

## Devolver un puntero a función

Podemos devolver un puntero a función desde una función. Por ejemplo:

```
int (*crear_puntero_suma())(int, int) {
    return suma;
}

int main() {
    int (*puntero_suma)(int, int) = crear_puntero_suma();
    int resultado = (*puntero_suma)(2, 3);
    printf("El resultado es: %d\n", resultado);
    return 0;
}
```

## Callbacks y punteros a funciones

Un *callback* es una función que se pasa como argumento a otra función, para que esta última pueda llamar a la primera función en un momento determinado. Los *callbacks* son una forma de implementar la programación orientada a eventos, donde una función puede notificar a otra función cuando sucede un evento específico.

### Punteros a funciones y *callbacks*

En C, los punteros a funciones son la base para implementar *callbacks*. Al pasar un puntero a función como argumento a otra función, podemos hacer que esta última función llame a la primera función en un momento determinado.

#### Ejemplo básico de *callback*

Supongamos que tenemos una función `imprimir_mensaje` que recibe un mensaje y lo imprime en la consola:

```
void imprimir_mensaje(const char *mensaje) {
    printf("%s\n", mensaje);
}
```

Ahora, supongamos que queremos crear una función `llamar_callback` que recibe un puntero a función y un mensaje, y llama a la función pasada como argumento con el mensaje:

```
void llamar_callback(void (*callback)(const char *), const char *mensaje) {
    callback(mensaje);
}
```

Podemos utilizar la función `llamar_callback` de la siguiente manera:

```
int main() {
    llamar_callback(imprimir_mensaje, "Hola, mundo!");
    return 0;
}
```

En este ejemplo, la función `llamar_callback` recibe un puntero a función `imprimir_mensaje` y un mensaje "Hola, mundo!". Luego, llama a la función `imprimir_mensaje` con el mensaje pasado como argumento.

#### Ejemplo con un *callback* más complejo

Supongamos que tenemos una función `procesar_datos` que recibe un array de enteros y un *callback* que se llama para cada elemento del array:

```
void procesar_datos(int *array, int tam, void (*callback)(int)) {
    for (int i = 0; i < tam; i++) {
        callback(array[i]);
    }
}
```

Podemos definir un *callback* que imprima cada elemento del array:

```
void imprimir_elemento(int elemento) {
    printf("%d\n", elemento);
}
```

Y utilizar la función `procesar_datos` de la siguiente manera:

```
int main() {
    int array[] = {1, 2, 3, 4, 5};
    int tam = sizeof(array) / sizeof(array[0]);
    procesar_datos(array, tam, imprimir_elemento);
    return 0;
}
```

# Punteros a void

En C, cuando se trabaja con *callbacks*, es común utilizar funciones que toman como argumento un puntero a **void**, es decir, **void \***. Esto se debe a que el tipo **void \*** es un tipo genérico que puede apuntar a cualquier tipo de dato.

## Estructuras de datos más complejas

Supongamos que tenemos una estructura de datos más compleja, como una estructura que representa un punto en un espacio 2D:

```
typedef struct {
    int x;
    int y;
} Punto;
```

## Pasar estructuras de datos a funciones que toman **void \***

Para pasar una estructura de datos como **Punto** a una función que toma un **void \*** como argumento, debemos utilizar un casting explícito. Por ejemplo:

```
void procesar_punto(void *dato) {
    Punto *punto = (Punto *)dato;
    printf("x: %d, y: %d\n", punto->x, punto->y);
}

int main() {
    Punto punto = {1, 2};
    procesar_punto(&punto);
    return 0;
}
```

## Utilidad del puntero a **void \***

El puntero a **void \*** es útil en *callbacks* porque nos permite pasar cualquier tipo de dato a una función sin tener que conocer su tipo exacto. Esto es especialmente útil cuando se trabaja con bibliotecas o *frameworks* que requieren *callbacks* genéricos (como las llamadas al sistema del kernel).

Por ejemplo, si estamos trabajando con una biblioteca de interfaces de usuario que requiere un *callback* para procesar eventos del ratón, podemos pasar un puntero a una estructura que representa un evento del ratón sin tener que modificar la firma de la función callback:

```
typedef struct {
    int x;
    int y;
    int boton;
} EventoMouse;

void procesar_evento(void *evento) {
    EventoMouse *evento_mouse = (EventoMouse *)evento;
    printf("Evento de mouse en (%d, %d) con botón %d\n", evento_mouse->x, evento_mouse->y, evento_mouse->boton);
}

int main() {
    EventoMouse evento = {10, 20, 1};
    procesar_evento(&evento);
    return 0;
}
```

## 2. Makefile

# Tutorial de Makefile

Un Makefile es un archivo que contiene instrucciones para compilar y enlazar código C. En este tutorial, recordarás como crear un archivo Makefile básico para compilar un programa en C a partir de un archivo de cabecera (.h) y un archivo de implementación (.c), que te será muy útil para realizar las tareas del Módulo II de prácticas.

## Estructura básica de un fichero Makefile

Un fichero Makefile (o simplemente “un Makefile”) se compone de reglas que se aplican para compilar y enlazar el código. Cada regla tiene la siguiente estructura:

```
objetivo: dependencias
comando
```

Donde:

- **objetivo** es el nombre del archivo o regla que se quiere generar (por ejemplo, un ejecutable, o una regla para realizar una acción como borrar ficheros).
- **dependencias** son los archivos que se necesitan para generar el objetivo (por ejemplo, archivos .c y .h).
- **comando** es la instrucción que se ejecuta para generar el objetivo.

## Ejemplo de Makefile

Supongamos que tenemos un archivo de cabecera `mi_programa.h` y un archivo de implementación `mi_programa.c`. Queremos compilar y enlazar estos archivos para generar un ejecutable llamado `mi_programa`.

Aquí está el Makefile:

```
mi_programa: mi_programa.o
gcc -o mi_programa mi_programa.o

mi_programa.o: mi_programa.c mi_programa.h
gcc -c mi_programa.c
```

Explicación:

- La primera regla dice que el objetivo es `mi_programa` y depende de `mi_programa.o`. El comando es `gcc -o mi_programa mi_programa.o`, que enlaza el archivo objeto `mi_programa.o` para generar el ejecutable `mi_programa`.
- La segunda regla dice que el objetivo es `mi_programa.o` y depende de `mi_programa.c` y `mi_programa.h`. El comando es `gcc -c mi_programa.c`, que compila el archivo `mi_programa.c` para generar el archivo objeto `mi_programa.o`.

Si solamente tenemos un único fichero `.c`, también podemos hacerlo todo en una única línea:

```
mi_programa: mi_programa.c mi_programa.h
gcc -o mi_programa mi_programa.c
```

## Regla clean

Para borrar los archivos objeto, podemos agregar una regla llamada `clean`. En general, podemos llamar a nuestras reglas con el nombre que queramos, pero es típico usar ciertos nombres como `clean`, `mrproper` (puedes leer más acerca de estas reglas [aquí](#)), etc.:

```
clean:
rm -f *.o
```

Esta regla elimina todos los archivos con extensión `.o` en el directorio actual.

## Uso del Makefile

Para compilar y ejecutar el programa, ejecuta los siguientes comandos:

```
$ make
$ ./mi_programa
```

Para borrar los archivos objeto, ejecuta:

```
$ make clean
```